

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни «Об'єктно-орієнтоване програмування»

на тему:

«Розробка графічного редактора об'єктів»

Виконав:

студентка групи ІМ-55

Горбань А.О

Номер в списку: 6 (Ж = 6)

Перевірив:

Рекечинський Д.О

Київ - 2026

Завдання:

1. Створити у середовищі розробки новий проект з ім'ям Lab2
2. Скомпілювати проект і отримати виконуваний файл програми
3. Перевірити роботу програми. За потреби налагодити програму

4. Проаналізувати та прокоментувати результати та вихідний текст програми
5. Оформити звіт

Завдання згідно з варіантом (Ж = 6):

Масив вказівників на об'єкти:

- Розмірність динамічного масиву: 106 елементів ($N = 6 + 100$).
- Тип масиву: динамічний масив вказівників `Shape **pcshape`

«Гумовий» слід:

- Суцільна лінія синього кольору (`PS_SOLID, RGB(0, 0, 255)`), яка реалізована за допомогою режиму малювання `SetROP2(hdc, R2_NOTXORPEN)`

Прямокутник (`RectangleShape`):

- Спосіб введення: по двох протилежних кутах.
- Спосіб відображення: чорний контур із кольоровим заповненням.
- Колір заповнення: жовтий (`RGB(255, 255, 0)`)

Еліпс (`EllipseShape`):

- Спосіб введення: від центру до кута охоплюючого прямокутника.
- Спосіб відображення: чорний контур із білим заповненням (`RGB(255, 255, 255)`)

Позначка поточного об'єкта в меню:

- Галочка навпроти обраного інструмента в меню «Об'єкти», яка реалізована за допомогою функції `CheckMenuItem` під час обробки повідомлення `WM_INITMENUPOPUP`

Файл `shape.h`

```
#ifndef SHAPE_H
#define SHAPE_H

#include <windows.h>

class Shape {
protected:
    long xs1, ys1, xs2, ys2;
public:
    virtual ~Shape() {}
};
```

```

        virtual void Set(long x1, long y1, long x2, long y2) {

            xs1 = x1; ys1 = y1; xs2 = x2; ys2 = y2;

        }

        virtual void Show(HDC hdc) = 0;

};

#endif

```

Файл point_shape.h

```

#ifndef POINT_SHAPE_H
#define POINT_SHAPE_H

#include "shape.h"

class PointShape : public Shape {
public:

    virtual void Show(HDC hdc) override;

};

#endif

```

Файл point_shape.cpp

```

#include "point_shape.h"

void PointShape::Show(HDC hdc) {

    SetPixel(hdc, xs1, ys1, RGB(0, 0, 0));

}

```

Файл line_shape.h

```
#ifndef LINE_SHAPE_H
#define LINE_SHAPE_H

#include "shape.h"

class LineShape : public Shape {
public:
    virtual void Show(HDC hdc) override;
};

#endif
```

Файл line_shape.cpp

```
#include "line_shape.h"

void LineShape::Show(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    MoveToEx(hdc, xs1, ys1, NULL);
    LineTo(hdc, xs2, ys2);

    SelectObject(hdc, hOldPen);
    DeleteObject(hPen);
}
```

Файл `rect_shape.h`

```
#ifndef RECT_SHAPE_H
#define RECT_SHAPE_H

#include "shape.h"

class RectangleShape : public Shape {
public:
    virtual void Show(HDC hdc) override;
};

#endif
```

Файл `rect_shape.cpp`

```
#include "rect_shape.h"

#include <algorithm>

void RectangleShape::Show(HDC hdc) {
    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HBRUSH hBrush = CreateSolidBrush(RGB(255, 255, 0)); // Жовте
заповнення

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);

    Rectangle(hdc, std::min(xs1, xs2), std::min(ys1, ys2),
std::max(xs1, xs2), std::max(ys1, ys2));
```

```
SelectObject(hdc, hOldPen);

SelectObject(hdc, hOldBrush);

DeleteObject(hPen);

DeleteObject(hBrush);

}
```

Файл ellipse_shape.h

```
#ifndef ELLIPSE_SHAPE_H
#define ELLIPSE_SHAPE_H

#include "shape.h"

class EllipseShape : public Shape {
public:

    virtual ~EllipseShape() {}

    virtual void Show(HDC hdc) override;

};

#endif
```

Файл ellipse_shape.cpp

```
#include "ellipse_shape.h"

#include <cmath>

void EllipseShape::Show(HDC hdc) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));

    HBRUSH hBrush = CreateSolidBrush(RGB(255, 255, 255)); // Белая
```

заливка

```
HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc, hBrush);


long rx = std::abs(xs2 - xs1);

long ry = std::abs(ys2 - ys1);


Ellipse(hdc, xs1 - rx, ys1 - ry, xs1 + rx, ys1 + ry);


SelectObject(hdc, hOldPen);

SelectObject(hdc, hOldBrush);

DeleteObject(hPen);

DeleteObject(hBrush);

}
```

Файл resource.h

```
#ifndef RESOURCE_H

#define RESOURCE_H


#define IDR_MENU1 101

#define IDM_EXIT 102

#define IDM_POINT 103

#define IDM_LINE 104

#define IDM_RECT 105

#define IDM_ELLIPSE 106

#define IDM_ABOUT 107
```

```
#endif
```

Файл resource.rc

```
#include "resource.h"
```

```
#include <windows.h>
```

```
#pragma code_page(65001)
```

```
IDR_MENU1 MENU
```

```
BEGIN
```

```
POPUP "Файл"
```

```
BEGIN
```

```
    MENUITEM "Вихід", IDM_EXIT
```

```
END
```

```
POPUP "Об'єкти"
```

```
BEGIN
```

```
    MENUITEM "Крапка", IDM_POINT
```

```
    MENUITEM "Лінія", IDM_LINE
```

```
    MENUITEM "Прямокутник", IDM_RECT
```

```
    MENUITEM "Еліпс", IDM_ELLIPSE
```

```
END
```

```
POPUP "Довідка"
```

```
BEGIN
```

```
    MENUITEM "Про програму", IDM_ABOUT
```

```
END
```

```
END
```

Файл Lab2.cpp

```
#include <windows.h>

#include <cmath>

#include <algorithm>

#include "resource.h"

#include "shape.h"

#include "point_shape.h"

#include "line_shape.h"

#include "rect_shape.h"

#include "ellipse_shape.h"


static Shape **pcshape = NULL;

static int shapeCount = 0;

static const int MAX_SHAPES = 106;


static int currentShapeType = IDM_POINT;

static bool isDrawing = false;

static long xStart = 0, yStart = 0, xPrev = 0, yPrev = 0;


void DrawRubberBand(HDC hdc, int shapeType, long x1, long y1, long x2,
long y2) {

    HPEN hPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 255)); // Синя
    суцільна лінія

    HPEN hOldPen = (HPEN)SelectObject(hdc, hPen);

    HBRUSH hOldBrush = (HBRUSH)SelectObject(hdc,
    GetStockObject(NULL_BRUSH));
```

```
int oldROP = SetROP2(hdc, R2_NOTXORPEN);

switch (shapeType) {

case IDM_POINT:

    SetPixel(hdc, x1, y1, RGB(0, 0, 255));

    break;

case IDM_LINE:

    MoveToEx(hdc, x1, y1, NULL);

    LineTo(hdc, x2, y2);

    break;

case IDM_RECT:

    Rectangle(hdc, std::min(x1, x2), std::min(y1, y2), std::max(x1,
x2), std::max(y1, y2));

    break;

case IDM_ELLIPSE: {

    long rx = std::abs(x2 - x1);

    long ry = std::abs(y2 - y1);

    Ellipse(hdc, x1 - rx, y1 - ry, x1 + rx, y1 + ry);

    break;

}

}

SetROP2(hdc, oldROP);

SelectObject(hdc, hOldBrush);

SelectObject(hdc, hOldPen);
```

```

DeleteObject(hPen);

}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM
lParam) {

    switch (message) {

    case WM_CREATE:

        pcshape = new Shape*[MAX_SHAPES];

        for (int i = 0; i < MAX_SHAPES; ++i) pcshape[i] = NULL;

        break;

    case WM_INITMENUPOPUP: {

        HMENU hMenu = (HMENU)wParam;

        CheckMenuItem(hMenu, IDM_POINT, MF_BYCOMMAND |
(currentShapeType == IDM_POINT ? MF_CHECKED : MF_UNCHECKED));

        CheckMenuItem(hMenu, IDM_LINE, MF_BYCOMMAND | (currentShapeType
== IDM_LINE ? MF_CHECKED : MF_UNCHECKED));

        CheckMenuItem(hMenu, IDM_RECT, MF_BYCOMMAND | (currentShapeType
== IDM_RECT ? MF_CHECKED : MF_UNCHECKED));

        CheckMenuItem(hMenu, IDM_ELLIPSE, MF_BYCOMMAND |
(currentShapeType == IDM_ELLIPSE ? MF_CHECKED : MF_UNCHECKED));

        break;

    }

    case WM_COMMAND:

        switch (LOWORD(wParam)) {

            case IDM_POINT:

```

```
case IDM_LINE:

case IDM_RECT:

case IDM_ELLIPSE:

    currentShapeType = LOWORD(wParam);

    break;

case IDM_ABOUT:

    MessageBox(hWnd, L"Графічний редактор об'єктів (Лабораторна  
№2)\nВаріант Ж = 6", L"Про програму", MB_OK | MB_ICONINFORMATION);

    break;

case IDM_EXIT:

    DestroyWindow(hWnd);

    break;

}

break;

case WM_LBUTTONDOWN:

    isDrawing = true;

    xStart = LOWORD(lParam);

    yStart = HIWORD(lParam);

    xPrev = xStart;

    yPrev = yStart;

    SetCapture(hWnd);

    {

        HDC hdc = GetDC(hWnd);

        DrawRubberBand(hdc, currentShapeType, xStart, yStart,
```

```
xPrev, yPrev);

        ReleaseDC(hWnd, hdc);

    }

    break;

case WM_MOUSEMOVE:

    if (isDrawing) {

        HDC hdc = GetDC(hWnd);

        DrawRubberBand(hdc, currentShapeType, xStart, yStart,
xPrev, yPrev);

        xPrev = LOWORD(lParam);

        yPrev = HIWORD(lParam);

        DrawRubberBand(hdc, currentShapeType, xStart, yStart,
xPrev, yPrev);

        ReleaseDC(hWnd, hdc);

    }

    break;

case WM_LBUTTONDOWN:

    if (isDrawing) {

        HDC hdc = GetDC(hWnd);

        DrawRubberBand(hdc, currentShapeType, xStart, yStart,
xPrev, yPrev);

        ReleaseDC(hWnd, hdc);

        ReleaseCapture();

        isDrawing = false;
```

```

        long xEnd = LOWORD(lParam);

        long yEnd = HIWORD(lParam);

        if (shapeCount < MAX_SHAPES) {

            Shape* p = NULL;

            switch (currentShapeType) {

                case IDM_POINT: p = new PointShape(); break;

                case IDM_LINE: p = new LineShape(); break;

                case IDM_RECT: p = new RectangleShape(); break;

                case IDM_ELLIPSE: p = new EllipseShape(); break;

            }

            if (p) {

                p->Set(xStart, yStart, xEnd, yEnd);

                pcshape[shapeCount++] = p;

                InvalidateRect(hWnd, NULL, TRUE);

            }

        }

        break;

case WM_PAINT: {

    PAINTSTRUCT ps;

    HDC hdc = BeginPaint(hWnd, &ps);

    for (int i = 0; i < shapeCount; ++i) {

```

```

        if (pcshape[i]) {

            pcshape[i]->Show(hdc);

        }

    }

    EndPaint(hWnd, &ps);

    break;

}

case WM_DESTROY:

    if (pcshape) {

        for (int i = 0; i < shapeCount; ++i) {

            delete pcshape[i];

        }

        delete[] pcshape;

    }

    PostQuitMessage(0);

    break;

default:

    return DefWindowProc(hWnd, message, wParam, lParam);

}

return 0;

}

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR

```

```
lpCmdLine, int nCmdShow) {

    WNDCLASSEX wc = { sizeof(WNDCLASSEX), CS_HREDRAW | CS_VREDRAW,
WndProc, 0, 0,

        hInstance, NULL, LoadCursor(NULL, IDC_ARROW),

        (HBRUSH) (COLOR_WINDOW + 1),
MAKEINTRESOURCE(IDR_MENU1),

        L"Lab2Class", NULL };

    if (!RegisterClassEx(&wc)) return 0;

    HWND hWnd = CreateWindow(L"Lab2Class", L"ООП_lab2 (Графічний
редактор)",

        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT,
CW_USEDEFAULT,

        800, 600, NULL, NULL, hInstance, NULL);

    if (!hWnd) return 0;

    ShowWindow(hWnd, nCmdShow);

    UpdateWindow(hWnd);

    MSG msg;

    while (GetMessage(&msg, NULL, 0, 0)) {

        TranslateMessage(&msg);

        DispatchMessage(&msg);

    }

    return (int)msg.wParam;
```

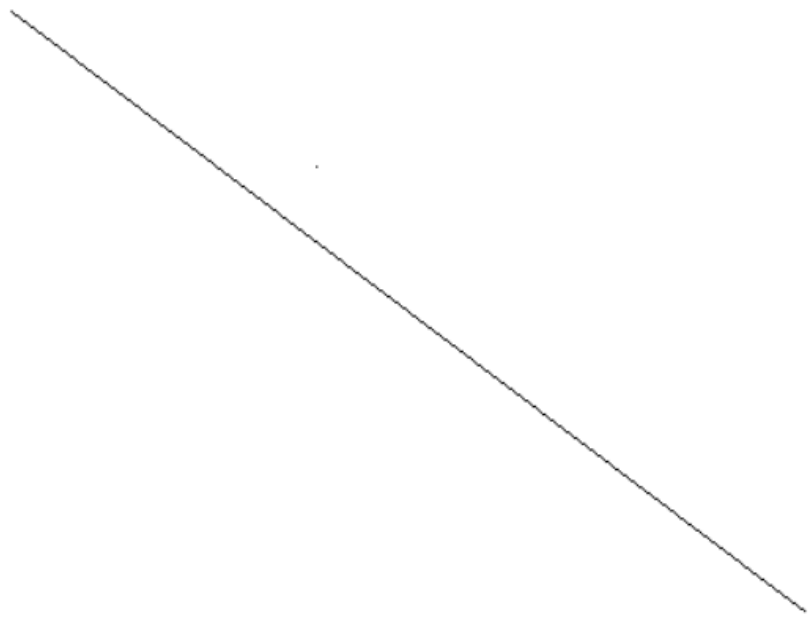


Скріншоти:



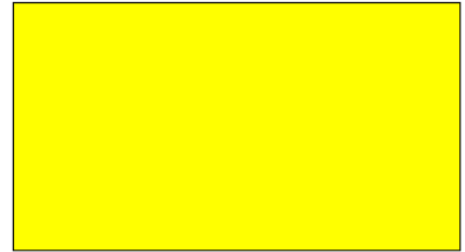
Крапка:

Лінія:

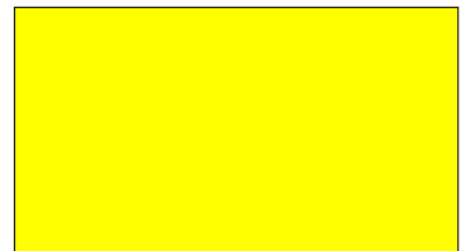
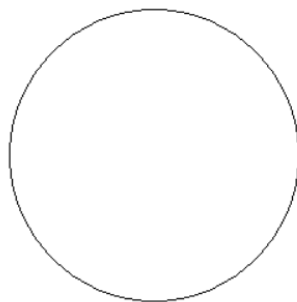
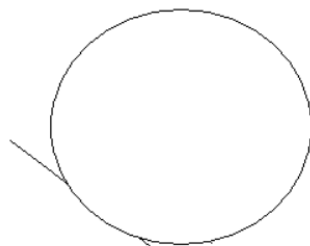


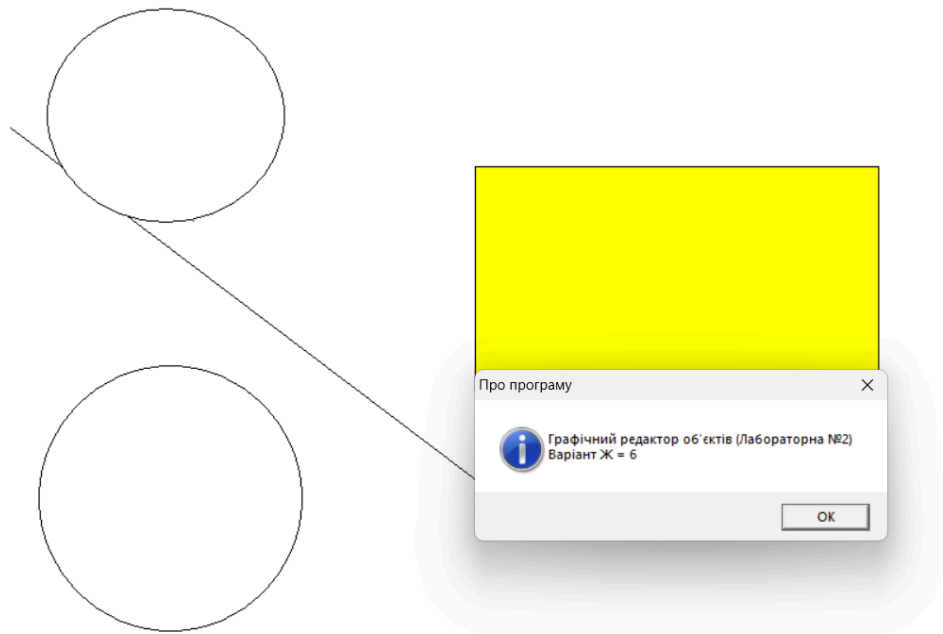
Прямокутник:

- Кратка
- Лінія
- ✓ Прямокутник
- Еліпс

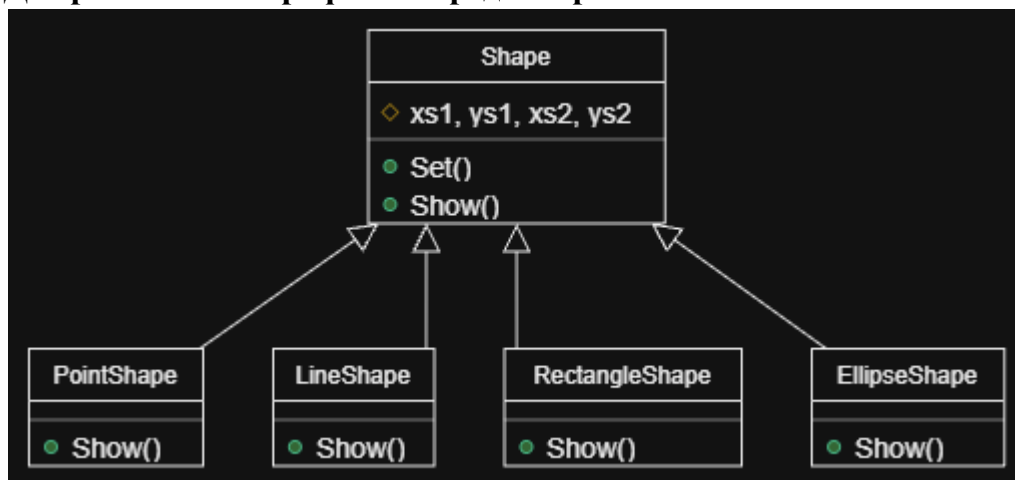


Еліпс:





Діаграма класів графічного редактора об'єктів:



Висновки:

Під час виконання лабораторної роботи №2 було розроблено векторний графічний редактор об'єктів на основі об'єктно-орієнтованого підходу та Win32 API

- Створено абстрактний базовий клас **Shape** із чисто віртуальним методом **Show**, від якого успадковано 4 похідні класи для конкретних графічних примітивів: **PointShape**, **LineShape**, **RectangleShape** та **EllipseShape**
- Реалізовано динамічний масив вказівників **pcshape** ємністю 106 елементів для збереження створених об'єктів згідно з Варіантом Ж = 6
- Забезпечено функціонал динамічного відображення синього «гумового» сліду за допомогою режиму малювання **R2_NOTXORPEN** під час розтягування фігур мишею
- Реалізовано алгоритми малювання жовтого прямокутника по

двох протилежних кутах та білого еліпса від центру до кута охоплюючого прямокутника

- Закріплено практичні навички застосування принципів інкапсуляції, успадкування, абстракції та поліморфізму в середовищі C++ при роботі з графічною підсистемою Win32 GDI